

Aula 11 — KNN e Árvores de Classificação

Curso de Aprendizado de Máquina — UFRJ

Leitura recomendada: AME §8.3–8.6; ISLP §4.7.6 e cap. 8

O que você vai aprender

Ao final desta aula você deve ser capaz de:

- adaptar o KNN da regressão para classificação e reconhecê-lo como classificador *plug-in*;
- construir uma **árvore de classificação**, predizendo pela moda da folha;
- definir **Gini** e **entropia** como medidas de impureza e explicar por que não se usa a taxa de erro para dividir;
- transferir *bagging*, florestas e *boosting* para classificação;
- comparar todos os classificadores do curso e escolher segundo a estrutura dos dados e o objetivo.

Esta é a última aula do Bloco II, e ela fecha um ciclo: praticamente tudo aqui é uma tradução do Bloco I, com a média virando *moda* e o EQM virando *impureza*. Comece pela pergunta:

*como uma árvore decide a melhor pergunta a fazer em cada nó,
quando o alvo é uma classe?*

A resposta ocupa a Seção 2.2, e o mais interessante nela é que a medida *óbvia* — a taxa de erro — é justamente a que não funciona.

1 KNN para classificação

Adaptação direta da regressão KNN (Aula 04): para classificar $\mathbf{x}$, olhe seus k vizinhos mais próximos e tome a **classe mais frequente** (voto da maioria):

$$\hat{g}(\mathbf{x}) = \text{moda}\{Y_i : i \in \mathcal{N}_{\mathbf{x}}\}. \quad (1)$$

Equivalentemente, é um classificador *plug-in* (Aula 08): estima-se

$$\hat{\mathbb{P}}(Y = c \mid \mathbf{x}) = \frac{1}{k} \sum_{i \in \mathcal{N}_{\mathbf{x}}} \mathbb{1}\{Y_i = c\}$$

(a proporção de vizinhos da classe c) e toma-se o $\arg \max_c$. Essa estimativa está, por construção, em $[0, 1]$.

Ideia central

A acurácia de treino de 1,000 com $k = 1$ é o exemplo mais limpo de todo o curso do que a Aula 01 dizia: o erro de treino é um estimador *otimista* do risco, e seu otimismo cresce com a flexibilidade. Aqui ele não é só otimista — é *perfeito*, e o modelo é o pior dos três. Se alguém lhe mostrar um classificador com 100% de acerto, a primeira pergunta é “em qual conjunto?”.

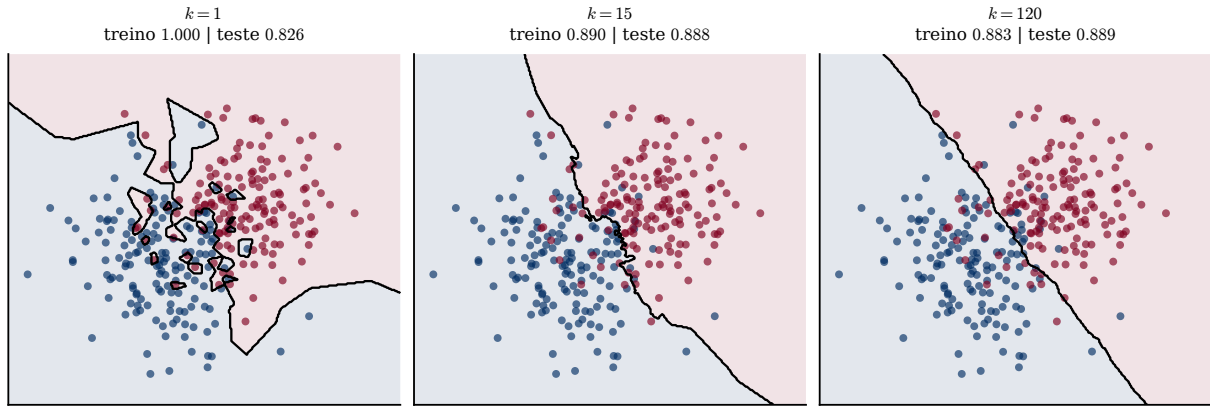


Figura 1: Fronteiras de decisão do KNN com $n = 300$ pontos de treino, avaliadas em 10 000 observações novas. Com $k = 1$ a fronteira é rendilhada e cria ilhas em torno de observações isoladas: a acurácia de treino é **exatamente** 1,000 — cada ponto é seu próprio vizinho mais próximo — e a de teste é a pior das três, 0,826. Com $k = 15$ a fronteira suaviza e o teste sobe para 0,888; com $k = 120$ ela fica quase reta e o teste é praticamente igual, 0,889. Compare com a Figura 1 da Aula 04: é o mesmo botão, com a mesma leitura invertida (k pequeno = flexível).

Atenção

Valem os mesmos cuidados da Aula 04: *padronize* as covariáveis (KNN usa distância) e lembre da *maldição da dimensionalidade* (Aula 05) — KNN não seleciona variáveis e degrada com muitas covariáveis irrelevantes. Em binário, use k *ímpar* para evitar empates.

2 Árvores de classificação

São o análogo das árvores de regressão (Aula 06): partição recursiva do espaço de covariáveis em regiões $R_1, \dots, R_J$. Duas adaptações:

2.1 Predição: a moda

Numa folha R_k , prediz-se a **classe majoritária** (em vez da média):

$$\hat{g}(\mathbf{x}) = \text{moda}\{Y_i : \mathbf{X}_i \in R_k\}, \quad \mathbf{x} \in R_k, \quad (2)$$

e $\hat{\mathbb{P}}(Y = c | \mathbf{x}) = \hat{p}_{R_k, c}$, a proporção da classe c na folha. É a substituição que o Teorema de Bayes da Aula 08 previa: em regressão o alvo era a média condicional, em classificação é a moda condicional.

2.2 Critério de divisão: impureza

O EQM não faz sentido com Y qualitativo. Em cada região R , seja $\hat{p}_{R, c}$ a proporção da classe c . Buscam-se divisões que tornem as folhas **puras** (idealmente, uma só classe por folha). Mede-se a impureza por:

$$\text{Índice de Gini: } G(R) = \sum_{c \in \mathcal{C}} \hat{p}_{R, c} (1 - \hat{p}_{R, c}), \quad (3)$$

$$\text{Entropia: } D(R) = - \sum_{c \in \mathcal{C}} \hat{p}_{R, c} \log \hat{p}_{R, c}. \quad (4)$$

Ambas são *mínimas* (zero) quando a folha é pura (algum $\hat{p}_{R, c} = 1$) e *máximas* quando as classes estão equilibradas. A cada passo escolhe-se a divisão que mais reduz a impureza (ponderada pelo tamanho das folhas).

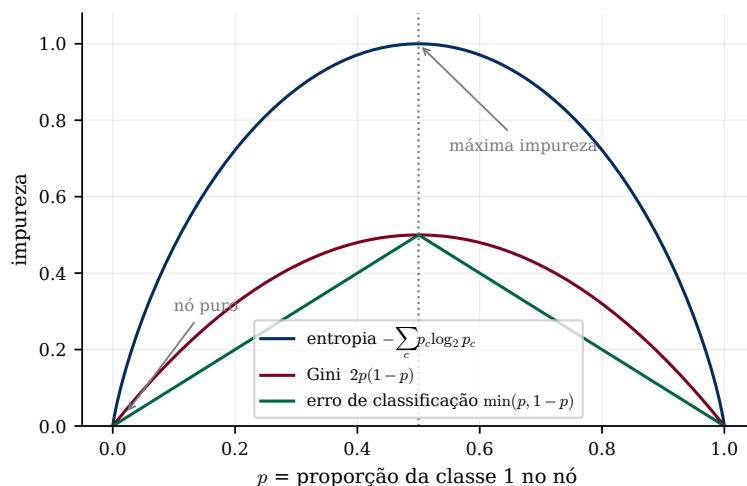


Figura 2: As três medidas num nó binário, em função da proporção p da classe 1. Todas zeram nos extremos (nó puro) e são máximas em $p = 1/2$. A diferença que importa está no *formato*: Gini e entropia são estritamente côncavas e suaves, enquanto o erro de classificação é feito de dois segmentos de reta — tem um bico em $p = 1/2$ e derivada constante nos dois lados.

Ideia central: Por que não usar a taxa de erro para dividir?

Olhe a curva verde da Figura 2: ela é *linear* em cada metade. Isso tem uma consequência devastadora. Suponha um nó com 400 observações, $p = 0,75$, e uma divisão que produza duas folhas de 200 cada, uma com $p = 0,50$ e outra com $p = 1,00$. O erro do pai é 0,25; o erro ponderado dos filhos é $\frac{1}{2}(0,50) + \frac{1}{2}(0,00) = 0,25$. **Redução zero** — e no entanto uma das folhas ficou *pura*, o que é claramente um progresso. Por linearidade, isso acontece sempre que a classe majoritária não muda. Gini e entropia, sendo estritamente côncavas, registram qualquer ganho de pureza: no mesmo exemplo o Gini cai de 0,375 para 0,250. A taxa de erro continua útil na *poda* final, onde o que interessa é justamente o desempenho de classificação.

Exemplo 2.1 (Escolha de divisão por Gini). Para decidir entre dividir por “Cardiologia” ou por “> 100 leitos” (predição: hospital tem tomógrafo), calcula-se o Gini ponderado de cada partição e escolhe-se a de *menor* valor (maior pureza). No exemplo do [AME], $\text{Gini}(\text{Leitos}) \approx 0,326 < \text{Gini}(\text{Cardiologia}) \approx 0,472$: divide-se por leitos.

Observação 2.2. Gini e entropia quase sempre escolhem as mesmas divisões — a diferença numérica entre as duas curvas da Figura 2 é pequena, e o *argmin* sobre as divisões candidatas raramente muda. Não vale a pena colocar essa escolha numa busca por validação cruzada; gaste o orçamento no α de poda ou na profundidade.

A construção segue as duas etapas da Aula 06: *crece-se* uma árvore grande e depois *poda-se* por custo-complexidade (α por CV). As mesmas vantagens valem: interpretabilidade, categóricas nativas, seleção automática de variáveis e captura de interações — e a mesma desvantagem, os cortes perpendiculares aos eixos.

3 Ensembles para classificação

Toda a Aula 06 se transfere; as árvores individuais agora classificam:

Bagging / Florestas

treina B árvores em amostras bootstrap e agrega por **voto da maioria** (ou

média das probabilidades — em geral melhor, porque preserva alguma informação sobre a confiança). Florestas descorrelacionam sorteando m variáveis por nó; regra empírica em classificação: $m \approx \sqrt{d}$, mais agressiva que o $d/3$ da regressão. Robustas a B ; OOB e importância de variáveis “de graça”.

Boosting ajuste sequencial. A variante clássica para classificação é o **AdaBoost**: a cada rodada, reponderam-se as observações dando mais peso às mal classificadas, e o classificador final é um voto ponderado dos aprendizes fracos. Como em regressão, B grande pode superajustar (use *early stopping*); XGBoost é a implementação de referência.

Observação 3.1. Vale registrar a ponte teórica: o AdaBoost, apesar de ter sido inventado por outro caminho, equivale a fazer *boosting* com a **perda exponencial** $e^{-y g(x)}$ — mais uma perda de margem, no mesmo eixo horizontal da Figura 4 da Aula 10. Vista assim, ela é ainda mais agressiva que a *hinge* com os pontos mal classificados, o que explica a sensibilidade conhecida do AdaBoost a rótulos errados.

4 Comparando os classificadores do curso

Método	Fronteira	Interpretab.	Observações
Logística	linear (no <i>log-odds</i>)	alta (coefs.)	dá probabilidades; robusta
LDA / QDA	linear / quadrática	média	ótimos sob gaussianidade
Bayes ingênuo	depende	média	forte com texto; descalibra se há correlação
KNN	muito flexível, local	baixa	sensível a escala e dimensão
Árvore	“escada” (eixos)	altíssima	instável sozinha
Florestas / Boosting	muito flexível	baixa	topo de desempenho em dados tabulares
SVM (kernel)	linear ou não linear	baixa	ótima com classes separadas

Ideia central

Não há método universalmente melhor (Aula 05: “não existe almoço grátis”). A escolha depende da *estrutura* dos dados e do *objetivo*. Quatro perguntas que resolvem a maioria dos casos:

1. **Preciso de probabilidades calibradas?** Se sim, logística; se não, qualquer um (Aula 09).
2. **Preciso explicar a decisão a alguém?** Se sim, árvore ou logística.
3. **Há muitas covariáveis irrelevantes?** Então evite KNN e SVM-RBF, que somam todas as coordenadas na distância; prefira Lasso, florestas ou *boosting* (Aula 05).
4. **Os dados são tabulares e o objetivo é acertar?** Comece por *boosting* de árvores — é o que costuma ganhar — e use os outros como referência.

E, em qualquer caso, compare os candidatos por validação cruzada com a métrica adequada ao problema.

5 Prática em Python

```

1 from sklearn.neighbors import KNeighborsClassifier
2 from sklearn.tree import DecisionTreeClassifier, plot_tree
3 from sklearn.ensemble import RandomForestClassifier, GradientBoostingClassifier
4
5 # KNN: k ímpar, padronizado dentro do pipeline (Aula 07)
6 knn = make_pipeline(StandardScaler(), KNeighborsClassifier(n_neighbors=11))
7
8 # Árvore: criterion "gini" (padrão) ou "entropy" -- na prática, tanto faz
9 arv = DecisionTreeClassifier(criterion="gini", ccp_alpha=0.01).fit(X_tr, y_tr)
10 plot_tree(arv, feature_names=nomes, class_names=classes, filled=True)
11
12 # Floresta: m = sqrt(d) e o padrão em classificação
13 rf = RandomForestClassifier(n_estimators=500, max_features="sqrt",
14                             oob_score=True, random_state=0).fit(X_tr, y_tr)

```

Repare que `max_features="sqrt"` já é o padrão do `RandomForestClassifier`, enquanto no `RandomForestRegressor` o padrão é usar todas as variáveis — isto é, *bagging*, não floresta. Se você quer uma floresta em regressão, precisa dizer.

Os notebooks *Comparação entre classificadores paramétricos* e *Exemplo - KNN e árvores em dados artificiais* desenharam as fronteiras de decisão de todos esses métodos lado a lado, nos mesmos dados. É o melhor jeito de fixar a intuição de “flexível × rígido” que percorreu o curso inteiro — vale rodar os dois.

Resumo

- **KNN (1)**: voto da maioria entre os k vizinhos; é *plug-in* com $\hat{\mathbb{P}}$ igual à proporção local. Com $k = 1$ o treino dá 100% e o teste dá o pior resultado (Fig. 1).
- **Árvore de classificação (2)**: prediz a moda da folha; divide minimizando **Gini (3)** ou **entropia (4)**.
- A *taxa de erro* não serve para dividir porque é linear por partes e registra redução zero em divisões que aumentam a pureza (Fig. 2); serve para podar.
- *Ensembles* transferem-se inteiros: florestas com $\bar{m} \approx \sqrt{d}$, AdaBoost como *boosting* com perda exponencial.
- Não há método universalmente melhor: escolha pela estrutura dos dados e pelo objetivo, e compare por validação cruzada com a métrica certa (Aula 09).

Leitura recomendada. [AME] §8.3 (árvores de classificação, com o exemplo do tomógrafo reproduzido aqui), §8.4–8.5 (*bagging*, florestas e *boosting* em classificação) e §8.6 (KNN). [ISLP] §4.7.6 (KNN) e Capítulo 8 — §8.1.2 trata especificamente das árvores de classificação e dos critérios de impureza, com a versão deles da nossa Figura 2.

Para praticar. [recursos/listas/Lista de exercícios 08.pdf](#).